

PRODUCT ARCHITECTURE

Avitus.

Version 2.9 / Agentic Remaster

An AI-native lead intelligence web app for property businesses. v2.9 keeps the v2.8 product surface intact and rebuilds the agentic architecture underneath it. The product remains an AI Lead Agent. The engineering becomes consolidated services, tiered approval, observable runs, an explicit Ask Avitus toolbelt, cost discipline, and a planned runtime substrate.

WHAT v2.9 LOCKS IN

- Five consolidated internal services. Twelve product-facing capabilities map onto them.
- Action Risk Tier model: Tier 0 silent, Tier 1 surface, Tier 2 prepare, Tier 3 external, Tier 4 restricted.
- Agent observability through agent_runs with owner correction capture for calibration.
- Ask Avitus toolbelt with typed tools, tenant checks, and tier-tagged outputs.
- Cost architecture: prompt caching, batch processing for imports, per-task model tiering.
- AgentContext object and versioned Prompt Packs per agent, vertical, and language.
- Runtime substrate roadmap: typed Edge Functions now, Inngest or Trigger.dev at V1.3.
- Failure-mode contract: success / partial / failed, plus a degraded-mode path when AI is unavailable.

WHAT CARRIES OVER FROM v2.8

- Positioning as an AI Lead Agent and lead intelligence web app for property businesses.
- Foundation, Signature, and Bespoke commercial tier model.
- Centralized orchestration over decentralized agent autonomy.
- Internal autonomy combined with owner-approved external actions. v2.9 makes the boundary explicit through tiers.
- Ask Avitus as a persistent lower-right assistant. v2.9 defines its toolbelt instead of leaving it implicit.
- Action Queue as the operational surface for what needs attention or approval now.
- Tenant isolation through canonical studio_id, RLS, and server-side ownership checks.
- AGENTS.md, SKILL.md, PRODUCT_BRIEF.md, and code_review.md remain implementation source of truth. This PDF is high-level architecture context only.

Executive Architecture Snapshot

v2.9 keeps the product surface stable and rebuilds the agentic spine for accuracy, cost, and durability.

Core principle. Avitus does the thinking and preparation automatically. Tier defines whether the owner sees nothing, sees an Action Queue item, sees a prepared draft, or is asked for approval before anything leaves the studio.

Product identity • AI Lead Agent and lead intelligence web app for property businesses. • Lead intelligence layer before or beside the CRM. • Turns messy lead information into clean, scored, actionable opportunities. • Specialized first for interior design studios and real estate teams.	Engineering identity • Five consolidated internal services behind twelve product-facing capabilities. • Central orchestrator coordinates server-side workflows. • Structured outputs, schema validation, deterministic scoring, and audit trails. • Action Risk Tiers control notification and approval per output.
Build now • Public intake form, Lead Inbox, Lead Detail, Paste Message, CSV import. • Lead cleanup, structured extraction, explainable scoring, missing info, follow-up draft. • Ask Avitus toolbelt foundation and Action Queue. • agent_runs observability table from day one.	Do not build yet • Direct Instagram DM, WhatsApp, email, or SMS sending. • Generic CRM bloat or full project management. • Decentralized agent autonomy or agents triggering external actions on their own. • Two-way sync before mappings, audit logs, and conflict rules are mature. • LangGraph, CrewAI, or LangChain in V1.

Strategic Shift v2.8 to v2.9

From a 12-agent taxonomy to a consolidated service spine with tiered approval and observable runs.

v2.8 architecture Twelve specialist agents described as separate services. Internal autonomy and external approval defined as a binary. Ask Avitus described as routing through the orchestrator. AI calls described as structured outputs with schema validation. Cost, observability, and runtime substrate left implicit.	v2.9 architecture Twelve product-facing capabilities preserved. Five consolidated internal services do the work underneath. Approval becomes a five-level tier. Ask Avitus has an explicit toolbelt. Every agent output goes through agent_runs. Cost has a defined discipline. Runtime substrate has a roadmap.
What stays the same • Centralized orchestration. Specialist services do not negotiate with each other. • Internal autonomy and external approval as the operating axis. • Tenant isolation through studio_id, RLS, and server-side ownership checks. • Ask Avitus as the owner-facing AI interface in the lower right of the web app.	What changes now • Internal services replace the 12-agent codepath sprawl. • Tiered approval replaces the binary internal / external split. • Ask Avitus tools are typed, named, and tenant-checked. • Cost, observability, runtime, and failure modes become first-class architecture concepts.

03 / ARCHITECTURE REFERENCE

Service Tier Model

Commercial packaging. v2.9 keeps the v2.8 tier model intact.

01 Foundation - Lead Inbox • Clean lead capture, tracking, and operational control. • Public intake form, Lead Inbox, Lead Detail, Paste Message, CSV import, basic mapping, custom fields, notes, status changes, reminders, statistics, won lead to project conversion. • Can be sold as a no-key pilot. • Tier 0 and Tier 1 outputs only by default.	02 Signature - AI Lead Agent • Everything in Foundation plus live AI extraction, summary, explainable scoring, missing information, suggested next action, and follow-up drafts. • Adds duplicate detection, stale lead detection, hot lead alerts, weekly reports, basic low-season detection, past-client check-ins. • Action Queue and Ask Avitus. • Tier 2 prepared drafts become available with owner click to send or copy.
03 Bespoke - Demand Engine • Everything in Signature plus CRM or Google Sheets sync, WhatsApp, email, calendar workflows. • Advanced routing, custom dashboards, team roles, approval-based sends. • Custom scoring, reactivation campaigns, referral outreach, prospecting, bespoke operating logic. • Tier 3 external sends and Tier 4 restricted bulk operations available with explicit guardrails.	Tier and risk relationship • Foundation runs in Tier 0 and Tier 1 only. • Signature unlocks Tier 2 prepared drafts. • Bespoke unlocks Tier 3 external sends with per-action approval, and Tier 4 restricted bulk operations with campaign approval. • Tier rules are enforced in the orchestrator, not at the service or UI layer.

04 / AGENTIC ARCHITECTURE

Action Risk Tier Model

v2.9 replaces the binary internal vs external split with a five-level tier. Every agent output is tagged.

Why tiers. The v2.8 binary made approval feel uniform when it is not. Marking a lead Lost without a message is different from sending a WhatsApp. Drafting follow-up is different from creating a reminder. Tiers let the orchestrator route uniformly instead of every service re-implementing approval logic.

Tier	Name	What it covers	Notification	Approval
0	Silent	Extract, score, classify, summarize, missing info detection.	None	None
1	Surface	Reminders, duplicate flags, stale and hot alerts, internal notes.	Action Queue	None
2	Prepare	Drafted follow-up, reactivation, check-in, or outreach text.	Action Queue	Explicit owner click to send or copy
3	External	Send WhatsApp, email, SMS, pricing, booking, disqualification, CRM final push, mark Lost when client-facing.	Action Queue	Per-action owner approval
4	Restricted	Bulk operations: mass reactivation, batch outreach, mass status changes, integration backfills.	Dedicated screen	Campaign approval, Bespoke only

Tier rules

- Every agent run is tagged with its produced tier.
- Tier rules are enforced in the orchestrator before persistence and before any side effect.
- A service cannot upgrade its own tier.
- Tier 3 and Tier 4 require an audit row even when the action is later cancelled.

Tier exceptions

- Tier 1 internal alerts may include the text of a Tier 2 prepared draft, but cannot send it.
- Tier 2 drafts can be edited by the owner before sending. Editing does not change the tier.
- Tier 4 is Bespoke only. Foundation and Signature studios see Tier 4 capabilities as locked features.

Consolidated Agent Services

Twelve product-facing capabilities. Five internal services underneath them.

Why consolidate. Several v2.8 agents (Nurture, Reactivation, Past Client, Prospecting) answer the same shape of question with different segments. Several others (Low-Season, Duplicate Detection) are mostly deterministic. Treating each as a separate codepath would mean twelve prompts, twelve eval surfaces, and twelve maintenance burdens.

Internal service	What it does	Product-facing capabilities
Lead Intelligence	Extract, score, classify, summarize, surface missing info, recommend next action.	Intake Cleanup, Qualification, Nurture (signal side)
Communication Drafting	One drafting engine. Different inputs produce different message shapes. Tone driven by Qualification Profile and brand voice.	Follow-Up, Reactivation, Past Client Check-In, Outbound Outreach
Pipeline Signals	Mostly deterministic detectors. LLM is used only as a tiebreaker for ambiguous cases.	Duplicate Detection, Stale Lead, Hot Lead, Low-Season
Reporting	Batch / scheduled runtime. Aggregates statistics, highlights movement, summarizes the week.	Reporting, weekly opportunity report
Integration Payloads	Schema-mapped payloads for external tools. Handles redaction, field mapping, and audit metadata.	Integration, Prospecting payloads, export jobs

Service boundary rules

- A service has a single typed input and a single typed output schema.
- Services do not call each other. The orchestrator chains them.
- Services do not write to the database directly. The orchestrator persists.
- Services do not produce side effects. They produce text and structured outputs that the orchestrator routes by tier.

Why product names stay

- The 12 product-facing names communicate value to studio owners. They appear in the UI, marketing, and Action Queue.
- The 5 internal services live only in the codebase, the agent_runs table, and this architecture document.
- A future service can be added without breaking the product vocabulary.

06 / AGENTIC ARCHITECTURE

AgentContext and Prompt Packs

Every service receives the same typed context. Prompts are versioned per agent, vertical, and language.

AgentContext is the single typed object the orchestrator hands every service. It carries tenant identity, vertical configuration, language, brand voice, prior agent outputs in the current workflow, and version pins for the schema and prompt pack. Without a single context type, services drift in what they expect, prompts duplicate setup logic, and per-vertical behavior becomes hard to audit.

AgentContext (TypeScript shape)

```
type AgentContext = {
  studio_id: string;
  vertical: 'interior_design' | 'real_estate';
  language: 'id' | 'en';
  qualification_profile: QualificationProfile;
  brand_voice: BrandVoice;
  currency: string;
  prior_outputs: AgentOutput[]; // earlier services in this workflow
  schema_version: string;
  prompt_pack_version: string;
  run_id: string; // links to agent_runs
  requested_tier: 0 | 1 | 2 | 3 | 4;
};
```

Prompt Pack model

- A Prompt Pack is a versioned bundle keyed by (service, vertical, language).
- Each pack contains: system prompt, few-shot examples, output schema, and tier policy.
- Packs are stored in the database and pinned per studio in Settings.
- A studio can be opted into a release channel: stable, beta, or pinned.

Why versioning matters

- Per-vertical tone and examples diverge meaningfully. Interior design follow-up is not real estate follow-up.
- Onboarding a new vertical (architecture, renovation, property management) ships a new pack, not new code.
- Rollbacks are explicit. A bad release rolls back the pack, not the deploy.
- Owner corrections (see Section 07) are joined back to prompt_pack_version for calibration.

07 / AGENTIC ARCHITECTURE

Agent Observability and Failure Modes

agent_runs is the spine. Every output is captured, every owner correction is captured, and every failure is graceful.

For an AI Lead Agent, accuracy is the product. v2.9 captures every agent run from V1.0 onward, even before live AI. The same table later becomes the eval set, the calibration signal, and the diagnostic surface when a studio reports the Qualification Agent feels miscalibrated.

agent_runs (logical shape)

```
agent_runs (  
  run_id, studio_id, lead_id, agent_name, service_name,  
  model, prompt_pack_version, schema_version,  
  input_hash, structured_output_jsonb, raw_text,  
  tier, status, confidence,  
  latency_ms, prompt_tokens, completion_tokens, cost_usd,  
  human_correction_jsonb, corrected_at, corrected_by,  
  created_at  
)
```

Run states

- success - schema validated, persisted, surfaced.
- partial - some fields valid, some missing or low confidence. Lead remains usable. Action Queue surfaces a review item.
- failed - schema invalid, model unreachable, or guardrail triggered. Lead remains usable through deterministic fallback.

Owner correction capture

- When an owner edits a draft, overrides a classification, or fixes an extracted field, the change is written to human_correction_jsonb on the originating run.
- This becomes the calibration dataset.
- Joined back to prompt_pack_version, this tells us when a pack regression happens.

Degraded mode

- If the AI provider is down or rate-limited, intake still works.
- Deterministic scoring runs against extracted fields where present.
- Missing fields are flagged.
- Action Queue gets a 'needs AI re-run' item once the provider returns.

Privacy posture

- Raw lead text is stored in raw_text on the run, never in logs.
- PII fields are extracted into the leads table where RLS applies.
- Logs carry run_id, studio_id, agent, status, latency, and cost. They do not carry phone numbers, emails, or message bodies.

Cost and Performance Architecture

Caching, batching, and per-task model tiering. Defined now so the unit economics survive Signature.

A 500-row CSV import calls services 1500 times. The Qualification Profile, scoring rubric, and few-shot examples are identical across every call. Without caching and batching, a single import can cost more than a month of inbound activity.

Prompt caching • Static prefix in every service prompt: Qualification Profile, scoring rubric, vertical examples, output schema, tier policy. • Cached at the provider where supported (Anthropic prompt caching, OpenAI cached input). • Variable suffix: lead text, prior outputs, tier request. • Cache key tied to prompt_pack_version so cache invalidates on pack release.	Batch processing • Bulk imports route through the Message Batches API. Not latency sensitive. Roughly half the cost of synchronous calls. • Inbound intake stays synchronous. The owner expects a fast Lead Inbox update. • Stale lead sweeps, weekly reports, and reactivation candidate scoring also batch.
Per-task model tiering • Cheap and fast model (Haiku-class) for: field extraction, duplicate tiebreak, classification refinement, summary generation. • Stronger model (Sonnet-class) for: follow-up drafting, reactivation drafting, weekly report narrative, Ask Avitus responses. • Reserved tier (Opus-class) only when explicitly justified. Not the default.	Cost ceilings and visibility • Per-studio monthly cost ceiling, configured per tier. • Cost per agent_run is logged. Per-studio dashboards in Settings show monthly burn and per-service split. • Soft cap warns the owner. Hard cap blocks Tier 2 and Tier 3 until reset or upgraded. • Imports show estimated cost before running.

Runtime Substrate Roadmap

Typed Edge Functions now. Durable workflows when the workload demands them. No agent framework before its time.

V1.0 to V1.1 substrate • Typed TypeScript pipeline inside Supabase Edge Functions. • Synchronous orchestrator. One workflow per request. • pg_cron handles light schedules: daily stale checks, weekly reports. • Sufficient for the inbound core, AI qualification, and first Ask Avitus capabilities.	Why this is enough early • Workloads are small. • Workflows are short and synchronous. • Failure modes are bounded. • A framework adds dependency and learning cost without earning its keep yet.
V1.3 substrate transition • Introduce Inngest or Trigger.dev when stale-lead sweeps, low-season detection, weekly reports, and large CSV imports all land at once. • Both are TypeScript-native and integrate cleanly with Supabase. • Brings durable retries, idempotency keys, fan-out and fan-in, and observable workflow state. • Does not require adopting LangGraph, CrewAI, or LangChain.	Frameworks deferred indefinitely • LangGraph, CrewAI, and LangChain are not on the roadmap for V1.0 through V2.0. • The orchestration model is deliberately simpler than what those frameworks are built for. • If a future Bespoke engagement requires durable multi-step agent workflows with human-in-the-loop branching, LangGraph becomes a candidate. Not before.

10 / AGENTIC ARCHITECTURE

Ask Avitus Toolbelt

v2.8 said Ask Avitus routes through the orchestrator. v2.9 defines exactly which tools it can call.

Ask Avitus is a tool-calling agent with a fixed toolbelt. It is not a planner that synthesizes new workflows on the fly, and it is not a generic RAG chatbot. Each tool is a typed Edge Function with RLS enforced and a tier tag. Ask Avitus produces tool calls. The orchestrator validates them, runs them, and returns results. Ask Avitus then composes a reply for the owner.

Tool	Inputs	Tier	Notes
query_leads	filters (status, classification, source, date range)	0	Read-only. Returns up to 50 lead summaries.
get_lead_detail	lead_id	0	Read-only. Full lead with score breakdown and history.
pipeline_metrics	window (7d, 30d, 90d)	0	Read-only. Counts, conversion rates, source performance.
explain_score	lead_id	0	Read-only. Returns deterministic breakdown plus AI summary.
find_duplicates	lead_id (optional, else studio-wide)	1	Surface only. Creates a review item, does not merge.
create_reminder	lead_id, due_at, note	1	Internal only. No client visibility.
prepare_followup_draft	lead_id, optional tone override	2	Saves a prepared draft. Owner click required to send or copy.
prepare_reactivation_draft	contact_id or lead_id, segment context	2	Same approval boundary as follow-up. Outbound consent required.
request_integration_export	scope (lead_ids), destination	3	Stages an export job. Final push requires owner approval.

Why a fixed toolbelt • Auditable. Every Ask Avitus action is one of a known set. • Bounded. Cannot exfiltrate data through clever phrasing. • Testable. Each tool has a known schema and known RLS behavior. • Upgradable. New capability = new tool, not new general intelligence.	Hard rules • Ask Avitus never bypasses studio_id, RLS, or tier rules. • Ask Avitus never executes Tier 3 or Tier 4 actions on its own. It can prepare them and surface them in the Action Queue. • Ask Avitus answers about a single studio's data only. • Ask Avitus declines requests it does not have a tool for.
--	---

11 / AGENTIC ARCHITECTURE

Orchestration Pattern

Centralized now. Hybrid later. Never decentralized in V1.

Recommended pattern. A Central Lead Intelligence Orchestrator coordinates the five consolidated services, applies tier policy, validates structured outputs, and persists results. Specialist services do not negotiate with each other.

Canonical V1 inbound pipeline

- Lead input
- > Orchestrator (loads AgentContext, prompt pack, schema)
 - > Lead Intelligence Service
 - > Schema validation
 - > Deterministic score calculation
 - > Communication Drafting Service (Tier 2)
 - > Pipeline Signals Service (duplicate check)
 - > Persist (lead, score breakdown, prepared draft)
 - > Write agent_runs entries
 - > Surface in Action Queue
 - > Owner reviews, edits, approves, or acts manually.

Use centralized orchestration now

- One orchestrator owns workflow order.
- Services perform narrow, typed tasks.
- Tier policy controls notification and approval.
- Database is the source of truth.

Avoid decentralized autonomy

- No peer-to-peer service negotiation in V1.
- No service-to-service decisions that bypass the orchestrator.
- No external sends or stage changes decided by a service alone.
- No framework runtime before the workload demands it.

Product Backbone

One web app core, multiple vertical templates, one AI interface, one operating spine.

Interface • Owners and teams review leads, actions, reminders, and opportunities. • V1 interface: Overview, Lead Inbox, Import, Intake Form, Projects, Settings, Action Queue, Ask Avitus.	Capture • Inbound leads enter through Public Intake Form, Paste Message, and CSV import. • These three paths prove the core workflow before direct channel integrations.
Data and configuration • Supabase / Postgres is the source of truth. • Canonical studio_id. RLS on every tenant-scoped table. • Behavior changes through Qualification Profile, Prompt Packs, vertical templates, scoring criteria, and custom fields.	AI intelligence and policy • Five consolidated services produce structured outputs and drafts. • Deterministic scoring weights and tier policy decide what happens next. • Owner approval is enforced by tier, not by individual service code.

13 / ARCHITECTURE REFERENCE

Lead Entry Architecture

Three controlled ways for inbound leads to enter Avitus.

Public Intake Form • Automatic. Prospect submits the Avitus form using a valid studio_slug. • The lead appears in the dashboard and is prepared for analysis, scoring, and next action. • Cleanest V1 workflow because it removes owner data entry.	Paste Message • Semi-manual. Owner pastes a WhatsApp, Instagram DM, email, SMS, referral, or website inquiry. • Lead Intelligence Service extracts fields and creates a reviewable lead. • Mobile-friendly. Many owners handle leads from their phone.
Import Sheet • Batch cleanup. Owner uploads CSV from Google Sheets. • Avitus detects columns, maps fields, preserves custom fields, imports leads, and prepares them for qualification. • Routes through the Message Batches API for cost.	Tier behavior at intake • Intake produces Tier 0 outputs by default: extracted fields, score, classification, summary. • Tier 2 prepared drafts only generate for Hot or Warm leads in Signature studios. • Tier 1 surface items appear when duplicates are likely or fields are missing.

Inbound and Outbound Growth Motions

Inbound first. Reactivation next. Property-specific prospecting last.

Inbound Lead Agent • Captures inquiries, cleans messy data, qualifies and scores leads. • Identifies missing information, recommends next action, drafts owner-approved follow-up. • Lifecycle: Capture, Clean, Qualify, Prioritize, Nurture, Prepare Follow-Up, Convert.	Outbound Opportunity Agent • Identifies prospects, surfaces stale leads, maintains past-client relationships. • Suggests referral outreach, detects low-season gaps, ranks opportunities, drafts owner-approved outreach. • Lifecycle: Identify, Research, Score, Segment, Draft Outreach, Owner Approves, Track Response, Nurture.
Outbound boundary • Outbound is not generic mass outreach. • Stale lead reactivation, past-client check-ins, referrals, low-season detection come before net-new prospecting. • All outbound drafts are Tier 2. All sends are Tier 3.	Drafting consolidation • Inbound follow-up, reactivation, past-client check-in, and outbound outreach are all produced by the Communication Drafting Service. • Differences come from input shape and brand voice context, not from separate codepaths.

Vertical Specialization Layer

Specialize through Prompt Packs, scoring weights, and form fields. Not through forks of the codebase.

Interior design studios • Qualification focus: project fit, scope clarity, budget fit, location fit, timeline feasibility, style preference, decision-maker readiness. • Lead fields: project type, property type, scope, style, area, budget, timeline, location, decision maker. • Drafting tone: design-led, consultative, scope-clarifying.	Real estate teams • Qualification focus: transaction intent, buyer/seller/renter/investor type, property type, location, budget or value, financing readiness, viewing readiness. • Lead fields: lead type, property type, preferred location, budget or value, financing/KPR/cash status, viewing availability. • Drafting tone: transactional, urgency-aware, viewing-oriented.
Specialization mechanism • During onboarding, the studio chooses Interior Design Studio or Real Estate Agency / Team. • Avitus selects the matching Prompt Pack and form template. • Scoring weights and Action Queue recommendations adjust to the vertical. • Ask Avitus context is filtered by vertical.	Onboarding new verticals • Architecture, renovation, property management, luxury consultancy can be added later. • Each ships as a Prompt Pack plus a form template plus a scoring weight set. • The codebase does not change. The product expands by configuration.

Seasonal Opportunity Layer

Detect slow periods. Recommend approved opportunity plays.

Signals tracked • Lead volume, qualified leads, hot leads waiting. • Consultation or viewing activity, won and lost movement. • Follow-up activity, lead source performance, stale lead count. • Enough-history check before drawing conclusions.	Initial detection logic • Deterministic before machine learning. • Example: qualified leads down 30 to 40 percent versus the prior 30, 60, or 90 day average for two or more weeks. • If history is too short, show: Not enough lead history yet. • The Pipeline Signals Service handles detection.
Recommended actions • Reactivate stale warm leads. • Check in with past clients. • Ask for referrals. • Revisit lost proposals. • Contact old consultations. • Prepare a property-specific prospecting list.	Required explanation • Why now. • Why this contact or segment. • What action is suggested. • What message should be drafted. • Whether owner approval is required.

Operating Autonomy Model

Internal autonomy. Tier-based external approval. v2.9 makes the boundary explicit through the tier model in Section 04.

Safe to automate (Tier 0 and Tier 1) • Create lead records. • Extract and clean fields. • Score, classify, summarize. • Detect duplicates, stale leads, hot leads. • Create reminders and Action Queue items. • Prepare reports and low-season recommendations.	Owner click required (Tier 2) • Send a prepared follow-up draft. • Send a prepared reactivation draft. • Send a prepared past-client check-in. • Stage an integration export. • Tier 2 outputs are saved as prepared actions. They are not sent until the owner clicks send or copy.
Per-action approval (Tier 3) • Send WhatsApp, email, SMS, pricing, booking link. • Mark a lead Lost when the action is client-facing. • Disqualify a lead. • Push to CRM as a final workflow action. • Trigger an external workflow that contacts a client.	Restricted (Tier 4) • Bulk reactivation across a segment. • Batch outreach. • Mass status changes or integration backfills. • Bespoke only. Requires campaign approval, not just per-action approval.

Action Queue

The operational surface for what needs attention or approval now.

Primary question • What needs my approval or attention now? • The Action Queue is more operational than a dashboard and more controlled than an autonomous sender.	Core items • Hot leads waiting. • Follow-ups due. • Prepared drafts. • Stale leads needing action. • Duplicates needing review. • Rows or imports needing review. • Missing information requests.
Opportunity items • Reactivation opportunities. • Past-client check-ins. • Referral prompts. • Low-season recommendations. • Weekly report recommendations. • Property-specific prospecting candidates for review.	Ask Avitus integration • Ask Avitus can explain any Action Queue item. • Ask Avitus can prepare drafts and create reminders against an item. • Ask Avitus cannot execute Tier 3 or Tier 4 actions on its own.

Explainable Scoring

Controlled qualification. Deterministic rules decide final points wherever possible. AI interprets text. The score is not a black box.

Interior design lead score • Budget fit: 30 points. • Timeline fit: 20 points. • Location fit: 15 points. • Project type fit: 15 points. • Decision-maker readiness: 10 points. • Clarity and completeness: 10 points.	Real estate lead score • Budget / property value fit: 25 points. • Timeline urgency: 20 points. • Location fit: 15 points. • Lead intent clarity: 15 points. • Financing / readiness: 15 points. • Contact completeness: 10 points.
Classifications • Hot, Warm, Cold, Not Fit, Needs Review. • Visible in the Lead Inbox, Lead Detail, Ask Avitus answers, and Action Queue, with reasons.	Scoring as a service responsibility • Lead Intelligence Service produces extracted fields and a classification suggestion. • The orchestrator computes the deterministic score from those fields and the studio's weights. • Disagreement between the two is logged on agent_runs as a calibration signal.

Data and Configuration Model

Flexible structure without becoming another messy spreadsheet.

Core lead fields • Name, contact, source, project or property type, location, budget, timeline, status, score, AI summary, recommended action, next follow-up. • Power the Lead Inbox, Lead Detail, scoring, filtering, statistics, reminders, Ask Avitus, and Action Queue.	Custom fields • Designer assigned, consultation date, payment status, quotation stage, financing status, viewing availability, agent assigned, and other client-specific fields. • Preserved without cluttering the main inbox.
Raw and archived data • Original notes, messy imported columns, legacy fields, unclear labels, old spreadsheet remarks. • Kept for traceability and search. Not centered in the main UI. Not exposed unnecessarily through Ask Avitus.	Studio Qualification Profile • Business name and type. Currency. Target budget range. • Preferred project or property types. Preferred locations. • Ideal client. Low-fit warning signs. Market focus. Signature styles. • Follow-up tone. Intake form intro. Thank-you message. • The Profile is an input to every Communication Drafting Service call.

Tenant Isolation and Source of Truth

Security rules that survive every implementation.

Tenant and access rules • Canonical tenant key: studio_id. • Supabase RLS on every tenant-scoped table. • Server-side ownership verification before tenant-scoped reads or writes. • Frontend filtering is never the tenant boundary.	AI and logging rules • Validate model output against schema before saving. • Leave the lead usable if AI fails (degraded mode). • Do not log raw lead messages, full CSV rows, phone numbers, emails, or secrets. • agent_runs.raw_text is the only place message text lives.
Ask Avitus rules • Active studio_id and user permissions on every tool call. • Cross-tenant data is unreachable. • No direct mutation of sensitive records outside approved server-side workflows. • No raw-data leakage in answers.	External actions • Every external send validates tenant ownership. • Every external integration uses explicit field mappings and produces an audit trail. • Tier 3 and Tier 4 actions cannot fire without a matching approval row.

Implementation Roadmap

Build the product core. Then activate AI. Then mature operations.

V1.0 Foundation Core / No-Key Pilot • Auth, tenant-scoped database, intake form, Lead Inbox, Lead Detail, basic settings, demo seed only. • Paste Message flow, notes, status, reminders, statistics, won lead to project conversion. • agent_runs table provisioned. Tier model wired. Action Queue skeleton. Ask Avitus stub.	V1.1 AI Activation • Lead Intelligence Service live. Structured extraction, score breakdown, classification, missing info, recommended action. • Communication Drafting Service for follow-up. • First Prompt Packs for interior design and real estate. • Ask Avitus toolbelt: query_leads, get_lead_detail, explain_score, pipeline_metrics. • Prompt caching on the static prefix.
V1.2 to V1.3 Import and Follow-Up Intelligence • CSV upload, column preview, mapping, custom field preservation. • Pipeline Signals Service: duplicates, stale leads, hot leads. • Follow-up reminders, weekly reports, internal alerts. • Message Batches API for imports. • Inngest or Trigger.dev introduced for durable workflows.	V1.4 to V2.0 • V1.4: Seasonal and reactivation. Communication Drafting Service for reactivation and check-ins. Tier 2 expansion. • V1.5: Integration Foundation. Google Sheets export, webhooks, Make.com bridge. Tier 3 enabled. • V2.0: Bespoke Demand Engine. CRM push, WhatsApp and email workflows, calendar, routing, approval-based sends, property-specific prospecting. Tier 4 unlocked.

Recommended Stack

Speed now. Portability later. No premature framework adoption.

Frontend / app - Existing codebase generated from Lovable, hardened with Codex. - React, Vite, TypeScript. - Lower-right Ask Avitus assistant. - Database-backed flows. No production mock fallbacks.	Database and auth - Supabase Postgres and Supabase Auth. - Canonical studio_id. Enforced RLS. Server-side ownership checks. - agent_runs as the observability spine. - Prompt Pack table for versioned per-vertical packs.
AI layer - Provider with structured outputs (Anthropic Claude or OpenAI), server-side only. - Prompt caching on the static prefix. - Message Batches API for bulk imports. - Per-task model tiering: Haiku-class for extraction, Sonnet-class for drafting. - API keys never in frontend code.	Hosting and runtime - Vercel or similar for the web app. - Supabase Edge Functions for V1.0 to V1.1 orchestration. - Inngest or Trigger.dev introduced at V1.3 for durable workflows. - Make.com retained as a Bespoke integration bridge after Signature is stable.

Product Principles and Final Positioning

What Avitus protects as it grows.

Build toward • Lead cleanup, qualification, explainable scores, AI summaries, suggested next actions. • Prepared follow-ups, nurturing, stale lead reactivation, past-client maintenance. • Seasonal opportunity detection, property-specific prospecting, custom fields, import / export. • Owner approval, vertical Prompt Packs, Ask Avitus toolbelt, Action Queue, Integration Hub.	Avoid in V1 • CRM bloat and full project management. • Direct Instagram DM integration or WhatsApp sending. • Premature two-way Sheets sync. • Uncontrolled AI outreach or black-box scoring. • LangChain, LangGraph, CrewAI as V1 dependencies. • Generic mass cold outreach positioning.
Product test • A messy inquiry, intake submission, or messy spreadsheet goes in. • A clean, scored, actionable lead record comes out, with explainable scoring, Ask Avitus answers, internal alerts, and owner-approved follow-up. • If yes, the core product exists.	Final positioning • Avitus is an AI-native lead intelligence web app for property businesses. • It qualifies and nurtures inbound leads, detects seasonal slowdowns, reactivates old opportunities, maintains past-client relationships, and identifies property-specific outbound prospects. • It routes clean lead intelligence into the tools teams already use, while keeping client-facing actions owner-approved.